Правительство Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
(НИУ ВШЭ)

Московский институт электроники и математики им. А.Н.Тихонова

ОТЧЁТ
О ПРАКТИЧЕСКОЙ РАБОТЕ №2
по дисциплине «Основы криптографии и стеганографии»
БЛОЧНЫЕ ШИФРЫ

Студент БИБ252
_____Мельников В.К.
«____»_____ 2026 г.

Руководитель
Заведующий кафедрой информационной
безопасности киберфизических систем
канд. техн. наук, доцент
_____О.О. Евсютин
«____»_____ 2026 г.

Москва 2026

СОДЕРЖАНИЕ

1 Задание на практическую работу 3

2 Краткая теоретическая часть 4

2.1 Описание шифров 4

2.2 Методы криптоанализа шифров 5

3 Примеры шифрования 6

4 Программная реализация шифров 9

5 Примеры криптоанализа 15

6 Выводы о проделанной работе 18

7 Список использованных источников 19

1 Задание на практическую работу

Целью данной работы является приобретение навыков программной реализации и криптоанализа блочных шифров.

В рамках практической работы необходимо выполнить следующее:

- Написать программную реализацию шифра Хилла и рекуррентного шифра Хилла
- Изучить методы криптоанализа блочных шифров с использованием дополнительных источников и провести криптоанализ
- Подготовть отчет о выолненной работе

2 Краткая теоретическая часть

2.1 Описание шифров

Шифр Хилла представляет собой пример блочного шифра, основанного на матричных преобразованиях с использованием арифметики остатков. Данный шифр устроен следующим образом. Открытый текст рассматривается как последовательность символов алфавита A можностью m, которые представляются элементами множества Z_m. Перед шифрованием текст разбивается на блоки фиксированной длины n. Каждый блок представляется в виде вектора-столбца P из n элементов. Ключом шифра является невырожденная квадратная матрица K размера $n \times n$ над полем Z_m. Процесс шифрования заключается в умножении матрицы ключа на вектор открытого текста по модулю m. Матрица будет обратима, если $\det(K) \neq 0$ и $\gcd(\det(K), m) = 1$. Формула нахождения обратной матрицы

$$K^{-1} = \det(K)^{-1} \cdot K \bmod m \tag{1}$$

Если длина открытого текста не кратна n, то он дополняется до нужной длины случайными символами. Процесс расшифрования осуществляется с помощью разбиения шифротекста на блоки длины n, представить каждый блок в виде вектора

$$Y = (y_1, y_2, ..., y_n)^T \tag{2}$$

После чего необходимо выполнить обратное умножение, получив блок открытого текста:

$$K^{-1} \cdot (y_1, y_2, ..., y_n)^T = X \tag{3}$$

$$X = (x_1, x_2, ..., x_n)^T \tag{4}$$

Рекуррентный шифр Хилла является расширением классического шифра Хилла, который использует уникальный ключ для каждого блока открытого текста. Принцип шифрования шифрования и расшифрования каждого отдельного блока аналогичен классическому шифру Хилла, но ключом будет являться две невырожденные матрицы K_1 и K_2 размера $n \times n$. Для шифрования первого блока открытого текста используется матрица K_1, для второго блока - K_2, для i-го блока

$$K_{i-1} \cdot K_{i-2} \bmod m \tag{5}$$

Для расшифрования и нахождения i-й матрица используется формула

$$K_i^{-1} = K_{i-2}^{-1} \cdot K_{i-1}^{-1} \tag{6}$$

Данный подход обеспечивает более высокий уровень безопасности по сравнению с классическим шифром Хилла, так как усложняет криптоанализ.

2.2 Методы криптоанализа шифров

Шифр Хилла и рекуррентный шифр Хилла устойчивы к обычному частотному анализу, так как открытый текст шифруется по блокам; разные буквы могут быть зашифрованы в одну и наоборот. Так же стоит отметить, что полный перебор ключей невозможен, если размер ключа равен хотя бы 3×3 и более. Количество возможных матриц $n \times n$ над полем Z_m равно m^{n^2}, для блока из $n = 2$ существует $26^4 = 456976$ матриц. Поскольку подойдут только невырожденные матрицы, то количество ключей будет меньше. Поэтому, если мы точно знаем, что $n = 2$, брутфорс возможен, но для $n = 3$ существует $26^9 > 5.4 \times 10^{12}$, доля обратимых матриц так же огромна. Для криптоанализа шифра Хилла и рекуррентного шифра Хилла можно использовать методы, основанные на анализе известных пар открытого текста и соответствующего ему шифротекста. Если известны n пар блоков открытого текста и соответствующих им блоков шифротекста, то можно составить систему линейных уравнений для определения ключевой матрицы. Для рекуррентного шифра Хилла необходимо иметь n^2 пар блоков открытого текста и соответствующих им блоков шифротекста для определения двух начальных ключевых матриц. [1]

3 Примеры шифрования

Пример шифрования с помощью шифра Хилла: Пусть алфавит состоит из 26 букв (a-z), мы шифруем слово «vladislav» с блоками длины 3. Выбираем ключевую матрицу

$$K = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}; \Delta K = 1 \cdot (-24) - 2 \cdot (-20) + 3 \cdot (-5) = 1 \tag{7}$$

Далее необходимо составить блоки открытого текста: «vla», «dis», «lav». Представляем их в виде векторов-столбцов $\mathrm{v} \to 21, \quad \mathrm{l} \to 11, \quad \mathrm{a} \to 0$ $\mathrm{d} \to 3, \quad \mathrm{i} \to 8, \quad \mathrm{s} \to 18$ $\mathrm{l} \to 11, \quad \mathrm{a} \to 0, \quad \mathrm{v} \to 21$

$$X_1 = \begin{pmatrix} 21 \\ 11 \\ 0 \end{pmatrix}, X_2 = \begin{pmatrix} 3 \\ 8 \\ 18 \end{pmatrix}, X_3 = \begin{pmatrix} 11 \\ 0 \\ 21 \end{pmatrix} \tag{8}$$

Зашифрованные блоки вычисляются следующим образом:

$$\begin{aligned} Y_1 &= \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} \begin{pmatrix} 21 \\ 11 \\ 0 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 21 \cdot 1 + 11 \cdot 2 + 3 \cdot 11 \\ 0 \cdot 21 + 1 \cdot 11 + 4 \cdot 0 \\ 5 \cdot 21 + 6 \cdot 11 + 0 \cdot 0 \end{pmatrix} (\mathrm{mod}\, 26) = \\ &= \begin{pmatrix} 43 \\ 11 \\ 171 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 17 \\ 11 \\ 15 \end{pmatrix} \\ Y_2 &= \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} \begin{pmatrix} 3 \\ 8 \\ 18 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 3 \cdot 1 + 8 \cdot 2 + 18 \cdot 3 \\ 0 \cdot 3 + 1 \cdot 8 + 4 \cdot 18 \\ 5 \cdot 3 + 6 \cdot 8 + 0 \cdot 18 \end{pmatrix} (\mathrm{mod}\, 26) = \\ &= \begin{pmatrix} 73 \\ 80 \\ 63 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 21 \\ 2 \\ 11 \end{pmatrix} \\ Y_3 &= \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} \begin{pmatrix} 11 \\ 0 \\ 21 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 11 \cdot 1 + 0 \cdot 2 + 21 \cdot 3 \\ 11 \cdot 0 + 0 \cdot 1 + 21 \cdot 4 \\ 11 \cdot 5 + 0 \cdot 6 + 21 \cdot 0 \end{pmatrix} (\mathrm{mod}\, 26) = \\ &= \begin{pmatrix} 74 \\ 84 \\ 55 \end{pmatrix} (\mathrm{mod}\, 26) = \begin{pmatrix} 22 \\ 6 \\ 3 \end{pmatrix} \end{aligned} \tag{9}$$

Таким образом, зашифрованный текст будет «rlpvclwgd».

Теперь расшифруем этот шифротекст. Для этого необходимо найти обратную матрицу к ключевой матрице K. Поскольку $\Delta K = 1$, то обратное число к ΔK по модулю 26 равно 1. Вычисляем матрицу алгебраических дополнений — $\mathrm{adj}(K)(\mathrm{mod}\, 26)$

$$\operatorname{adj}(K)(\operatorname{mod} 26) = \begin{pmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & -1 \end{pmatrix} (\operatorname{mod} 26) \Rightarrow K^{-1}(\operatorname{mod} 26) =$$
$$= \frac{1}{1} \cdot \begin{pmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & -1 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \tag{10}$$

Теперь можно посчитать расшифрованные блоки по формуле (3), но перед этим необходимо узнать индексы букв шифротекста: $r \to 17, l \to 11, p \to 15, v \to 21, c \to 2, l \to 11, w \to 22, g \to 6, d \to 3$

$$X_1 = \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \begin{pmatrix} 17 \\ 11 \\ 15 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 2 \cdot 17 + 18 \cdot 11 + 5 \cdot 15 \\ 20 \cdot 17 + 11 \cdot 11 + 22 \cdot 15 \\ 21 \cdot 17 + 4 \cdot 11 + 1 \cdot 15 \end{pmatrix} (\operatorname{mod} 26) =$$
$$= \begin{pmatrix} 307 \\ 791 \\ 416 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 21 \\ 11 \\ 0 \end{pmatrix}$$
$$X_2 = \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \begin{pmatrix} 21 \\ 2 \\ 11 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 2 \cdot 21 + 18 \cdot 2 + 5 \cdot 11 \\ 20 \cdot 21 + 11 \cdot 2 + 22 \cdot 11 \\ 21 \cdot 21 + 4 \cdot 2 + 1 \cdot 11 \end{pmatrix} (\operatorname{mod} 26) =$$
$$= \begin{pmatrix} 133 \\ 684 \\ 460 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 3 \\ 8 \\ 18 \end{pmatrix} \tag{11}$$
$$X_3 = \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \begin{pmatrix} 22 \\ 6 \\ 3 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 2 \cdot 22 + 18 \cdot 6 + 5 \cdot 3 \\ 20 \cdot 22 + 11 \cdot 6 + 22 \cdot 3 \\ 21 \cdot 22 + 4 \cdot 6 + 1 \cdot 3 \end{pmatrix} (\operatorname{mod} 26) =$$
$$= \begin{pmatrix} 167 \\ 572 \\ 489 \end{pmatrix} (\operatorname{mod} 26) = \begin{pmatrix} 11 \\ 0 \\ 21 \end{pmatrix}$$

Теперь можно расшифровать текст $21 \to v, 11 \to l, 0 \to a, 3 \to d, 8 \to i, 18 \to s, 11 \to l, 0 \to a, 21 \to v$, получили «vladislav»

Пример шифрования с помощью рекуррентного шифра Хилла:
Для этого нужно взять два начальных ключа и блок длины 3

$$K_1 = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}, K_2 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$
$$K_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} = \begin{pmatrix} 7 & 11 & 10 \\ 1 & 3 & 7 \\ 6 & 9 & 7 \end{pmatrix} \tag{12}$$
$$\Delta K_2 = 1 \Rightarrow \Delta K_3 = 1$$

Согласно (9) первый блок зашифруется как «rlp», поэтому можно начать с зашифровки второго блока.

$$
\begin{aligned}
Y_2 &= \begin{pmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 8 \\ 18 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 32 \\ 11 \\ 29 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 6 \\ 11 \\ 3 \end{pmatrix} \\
Y_3 &= \begin{pmatrix} 7 & 11 & 10 \\ 1 & 3 & 7 \\ 6 & 9 & 7 \end{pmatrix} \begin{pmatrix} 11 \\ 0 \\ 21 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 287 \\ 158 \\ 213 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}
\end{aligned} \tag{13}
$$

Таким образом, зашифрованный текст будет «rlpgldbcf»

Теперь расшифруем этот шифротекст. Для этого необходимо найти обратные матрицы к ключам K_1 и K_2. Поскольку $\Delta K_1 = 1$ и $\Delta K_2 = 1$, то обратные числа к ним по модулю 26 равны 1. Вычисляем матрицы алгебраических дополнений — $\mathrm{adj}(K_1)(\bmod 26)$, $\mathrm{adj}(K_2)(\bmod 26)$ и $\mathrm{adj}(K_3)(\bmod 26)$. Согласно (10)

$$
\begin{aligned}
K_1^{-1} &= \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \Rightarrow X_1 = \begin{pmatrix} 2 & 18 & 5 \\ 20 & 11 & 22 \\ 21 & 4 & 1 \end{pmatrix} \begin{pmatrix} 17 \\ 11 \\ 15 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 21 \\ 11 \\ 0 \end{pmatrix} \\
K_2^{-1} &= \begin{pmatrix} 1 & 0 & 25 \\ 25 & 1 & 1 \\ 0 & 25 & 1 \end{pmatrix} \Rightarrow X_2 = \begin{pmatrix} 1 & 0 & 25 \\ 25 & 1 & 1 \\ 0 & 25 & 1 \end{pmatrix} \begin{pmatrix} 6 \\ 11 \\ 3 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 3 \\ 8 \\ 18 \end{pmatrix} \\
K_3^{-1} &= \begin{pmatrix} 10 & 13 & 21 \\ 9 & 15 & 13 \\ 17 & 3 & 10 \end{pmatrix} \Rightarrow X_3 = \begin{pmatrix} 10 & 13 & 21 \\ 9 & 15 & 13 \\ 17 & 3 & 10 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix} (\bmod 26) = \begin{pmatrix} 11 \\ 0 \\ 21 \end{pmatrix}
\end{aligned} \tag{14}
$$

Получился «vladislav»

4 Программная реализация шифров

Программная реализация шифра Хилла и рекуррентного шифра Хилла на языке Python с использованием библиотеки NumPy для работы с матрицами, random для генерации случайных символов, если необходимо и math для нахождения НОД. Реализация включает функции для шифрования, расшифрования и нахождения обратной матрицы по модулю. Были реализованы вспомогательные методы: text_to_block разбивает открытый текст на блоки длины n, а matrix_to_key реализует удобный ввод матрицы ключа через консоль

```
class BlockCiphers:
    Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
    def __init__(self, alphabet):
        self.alphabet = alphabet
        self.m = len(alphabet)

    Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
    def text_to_block(self, text, block_size):
        word = [self.alphabet.index(char) for char in text]
        blocks = []
        for i in range(0, len(word), block_size):
            block = word[i:block_size + i]
            if len(block) < block_size:
                random_letters = [random.randint(0, self.m - 1) for _ in range(block_size - len(block))]
                block.extend(random_letters)
            blocks.append(block)

        return blocks

    Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
    def matrix_to_key(self, text):
        lines = text.strip().split('\n')
        key = []
        for line in lines:
            row = [int(num) for num in line.split()]      You, 23 hours ago • Hill Cipher + recurrent …
            key.append(row)
        return np.array(key)
```

Рисунок 1 — Реализация вспомогательных функций

Далее был реализован основной шифр Хилла.

```
    def hill_cipher_encrypt(self, text, block_size, key_text):
        key = self.matrix_to_key(key_text)
        blocks = self.text_to_block(text, block_size)

        if len(key.shape) != 2 or key.shape[0] != key.shape[1]:
            raise KeyValidityError("Матрица ключа должна быть квадратной (разное количество символов в словах).")

        if key.shape != (block_size, block_size):
            raise KeyValidityError(f"Ключ должен быть размера {block_size}x{block_size}.")

        det_float = np.linalg.det(key)
        det_int = int(round(det_float))

        if det_int == 0:
            raise KeyValidityError("Матрица ключа вырожденная (определитель равен 0).")

        if math.gcd(det_int % self.m, self.m) != 1:
            raise KeyValidityError("Определитель матрицы ключа не взаимно прост с мощностью алфавита.")

        encrypted_blocks = []
        for block in blocks:
            P = np.array(block).reshape(-1, 1)
            C = np.dot(key, P) % self.m
            encrypted_block = C.flatten().tolist()
            encrypted_blocks.extend(encrypted_block)

        encrypted_text = ''.join([self.alphabet[idx] for idx in encrypted_blocks])
        return encrypted_text
```

Рисунок 2 — Реализация шифрования Хилла

```python
def hill_cipher_decrypt(self, encrypted_text, block_size, key_text):
    key = self.matrix_to_key(key_text)
    encrypted_blocks = self.text_to_block(encrypted_text, block_size)

    if len(key.shape) != 2 or key.shape[0] != key.shape[1]:
        raise KeyValidityError("Матрица ключа должна быть квадратной (разное количество символов в словах).")

    if key.shape != (block_size, block_size):
        raise KeyValidityError(f"Ключ должен быть размера {block_size}x{block_size}.")

    det_float = np.linalg.det(key)
    det_int = int(round(det_float))

    if det_int == 0:
        raise KeyValidityError("Матрица ключа вырожденная (определитель равен 0).")

    if math.gcd(det_int % self.m, self.m) != 1:
        raise KeyValidityError("Определитель матрицы ключа не взаимно прост с мощностью алфавита.")

    det_mod = det_int % self.m
    try:
        det_inv = pow(det_mod, -1, self.m)
    except ValueError:
        raise KeyValidityError("Определитель матрицы ключа не имеет обратного по модулю (не взаимно прост с мощн
    adjugate_matrix = np.round(np.linalg.inv(key) * det_float).astype(int)
    key_inv = (det_inv * adjugate_matrix) % self.m
    decrypted_blocks = []
    for block in encrypted_blocks:
        C = np.array(block).reshape(-1, 1)
        P = np.dot(key_inv, C) % self.m
        decrypted_block = P.flatten().tolist()
        decrypted_blocks.extend(decrypted_block)

    decrypted_text = ''.join([self.alphabet[idx] for idx in decrypted_blocks])
    return decrypted_text
```

Рисунок 3 — Реализация расшифрования Хилла

Следующие две функции реализуют зашифрование и расшифрование рекуррентного шифра Хилла

```python
def recurrent_hill_cipher_encrypt(self, text, block_size, key_text_1, key_text_2):
        key1 = self.matrix_to_key(key_text_1)
        key2 = self.matrix_to_key(key_text_2)

        # Валидация первого ключа
        if len(key1.shape) != 2 or key1.shape[0] != key1.shape[1] or key1.shape != (block_size, block_size):
            raise KeyValidityError(f"Первый ключ должен быть квадратной матрицей {block_size}x{block_size}.")
        det1_int = int(round(np.linalg.det(key1)))
        if det1_int == 0 or math.gcd(det1_int % self.m, self.m) != 1:
            raise KeyValidityError("Определитель первого ключа вырожден или не взаимно прост с мощностью алфави

        # Валидация второго ключа
        if len(key2.shape) != 2 or key2.shape[0] != key2.shape[1] or key2.shape != (block_size, block_size):
            raise KeyValidityError(f"Второй ключ должен быть квадратной матрицей {block_size}x{block_size}.")
        det2_int = int(round(np.linalg.det(key2)))
        if det2_int == 0 or math.gcd(det2_int % self.m, self.m) != 1:
            raise KeyValidityError("Определитель второго ключа вырожден или не взаимно прост с мощностью алфави

        blocks = self.text_to_block(text, block_size)
        encrypted_blocks = []

        for i, block in enumerate(blocks):
            if i == 0:
                current_key = key1
            elif i == 1:
                current_key = key2
            else:
                current_key = np.dot(key2, key1) % self.m
                key1 = key2
                key2 = current_key

            P = np.array(block).reshape(-1, 1)
            C = np.dot(current_key, P) % self.m

            encrypted_block = [int(val) for val in C.flatten().tolist()]
            encrypted_blocks.extend(encrypted_block)

        encrypted_text = ''.join([self.alphabet[idx] for idx in encrypted_blocks])      You, 23 hours ago • Hil
        return encrypted_text
```

Рисунок 4 — Реализация шифрования рекуррентного шифра Хилла

```python
def recurrent_hill_cipher_decrypt(self, encrypted_text, block_size, key_text_1, key_text_2):
        key1 = self.matrix_to_key(key_text_1)
        key2 = self.matrix_to_key(key_text_2)

        if len(key1.shape) != 2 or key1.shape[0] != key1.shape[1] or key1.shape != (block_size, block_size):
            raise KeyValidityError(f"Первый ключ должен быть квадратной матрицей {block_size}x{block_size}."
        det1_int = int(round(np.linalg.det(key1)))
        if det1_int == 0 or math.gcd(det1_int % self.m, self.m) != 1:
            raise KeyValidityError("Определитель первого ключа вырожден или не взаимно прост с мощностью алф

        try:
            det1_inv = pow(det1_int % self.m, -1, self.m)
        except ValueError:
            raise KeyValidityError("Первый ключ не имеет обратного по модулю.")
        adj1 = np.round(np.linalg.inv(key1) * np.linalg.det(key1)).astype(int)
        key1 = (det1_inv * adj1) % self.m

        if len(key2.shape) != 2 or key2.shape[0] != key2.shape[1] or key2.shape != (block_size, block_size):
            raise KeyValidityError(f"Второй ключ должен быть квадратной матрицей {block_size}x{block_size}."
        det2_int = int(round(np.linalg.det(key2)))
        if det2_int == 0 or math.gcd(det2_int % self.m, self.m) != 1:
            raise KeyValidityError("Определитель второго ключа вырожден или не взаимно прост с мощностью алф

        try:
            det2_inv = pow(det2_int % self.m, -1, self.m)
        except ValueError:
            raise KeyValidityError("Второй ключ не имеет обратного по модулю.")
        adj2 = np.round(np.linalg.inv(key2) * np.linalg.det(key2)).astype(int)
        key2 = (det2_inv * adj2) % self.m

        encrypted_blocks = self.text_to_block(encrypted_text, block_size)
        decrypted_blocks = []

        for i, block in enumerate(encrypted_blocks):
            if i == 0:
                key_inv = key1
            elif i == 1:
                key_inv = key2
            else:
                key_inv = np.dot(key1, key2) % self.m
                key1 = key2
                key2 = key_inv

            # Расшифровываем блок: P = (K^-1 * C) mod m
            C = np.array(block).reshape(-1, 1)
            P = np.dot(key_inv, C) % self.m
            decrypted_block = P.flatten().tolist()
            decrypted_blocks.extend(decrypted_block)

        decrypted_text = ''.join([self.alphabet[idx] for idx in decrypted_blocks])
        return decrypted_text
```

Рисунок 5 — Реализация расшифрования рекуррентного шифра Хилла

В основной main функции реализовано консольное меню для удобного шифрования и расшифрования.

```python
if __name__ == "__main__":
    user_alphabet = input("Пропишите '1', если по умолчанию следует использовать латинский строчный алфавит.\nВведите алфавит: ")
    if user_alphabet == '1':
        app = BlockCiphers(string.ascii_lowercase)
    else:
        app = BlockCiphers(user_alphabet)

    text = input("Введите текст: ")
    block_size = int(input("Введите длину блока разбиения текста: "))

    cipher_type = input("Введите '1' для обычного шифра Хилла, '2' для рекуррентного: ")

    if cipher_type == '1':
        print(f"Введите матрицу ключа построчно (числа через пробел, нужно ввести {block_size} строк(и)):")
        key_rows = []
        for i in range(block_size):
            row = input(f"Строка {i + 1}: ")
            key_rows.append(row)
        key_text = "\n".join(key_rows)

        choice = input("Введите '1' для шифрования, '2' для расшифрования: ")
        if choice == '1':
            try:
                print("\nЗашифрованный текст:", app.hill_cipher_encrypt(text, block_size, key_text))
            except KeyValidityError as e:
                print(f"\nОшибка ключа: {e}")
        elif choice == '2':
            try:
                print("\nРасшифрованный текст:", app.hill_cipher_decrypt(text, block_size, key_text))
            except KeyValidityError as e:
                print(f"\nОшибка ключа: {e}")

    elif cipher_type == '2':
            print(f"Введите матрицу первого ключа построчно (числа через пробел, нужно ввести {block_size} строк(и)):")
            key_rows_1 = []
            for i in range(block_size):
                row = input(f"Строка {i + 1}: ")
                key_rows_1.append(row)
            key_text_1 = "\n".join(key_rows_1)

            print(f"Введите матрицу второго ключа построчно (числа через пробел, нужно ввести {block_size} строк(и)):")
            key_rows_2 = []
            for i in range(block_size):
                row = input(f"Строка {i + 1}: ")
                key_rows_2.append(row)
            key_text_2 = "\n".join(key_rows_2)

            action = input("Введите '1' для шифрования, '2' для расшифрования: ")
            try:
                if action == '1':
                    print("\nЗашифрованный текст:", app.recurrent_hill_cipher_encrypt(text, block_size, key_text_1, key_text_2))
                elif action == '2':
                    print("\nРасшифрованный текст:", app.recurrent_hill_cipher_decrypt(text, block_size, key_text_1, key_text_2))
            except KeyValidityError as e:
                print(f"\nОшибка ключа: {e}")
```

Рисунок 6 — Реализация консольного меню

Проверим корректность реализации на примерах, изложенных выше.

```
(.venv) dvvd@Debux:~/Documents/Crypto$ /home/dvvd/Documents/Crypto/.venv/bin/python /home/dvvd/Documents/Crypto/HillCipher/HillCipher.py
Пропишите '1', если по умолчанию следует использовать латинский строчный алфавит.
Введите алфавит: 1
Введите текст: vladislav
Введите длину блока разбиения текста: 3
Введите '1' для обычного шифра Хилла, '2' для рекуррентного: 1
Введите матрицу ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 1 2 3
Строка 2: 0 1 4
Строка 3: 5 6  0
Введите '1' для шифрования, '2' для расшифрования: 1

Зашифрованный текст: rlpvclwgd
(.venv) dvvd@Debux:~/Documents/Crypto$ /home/dvvd/Documents/Crypto/.venv/bin/python /home/dvvd/Documents/Crypto/HillCipher/HillCipher.py
Пропишите '1', если по умолчанию следует использовать латинский строчный алфавит.
Введите алфавит: 1
Введите текст: rlpvclwgd
Введите длину блока разбиения текста: 3
Введите '1' для обычного шифра Хилла, '2' для рекуррентного: 1
Введите матрицу ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 1 2 3
Строка 2: 0 1 4
Строка 3: 5 6 0
Введите '1' для шифрования, '2' для расшифрования: 2

Расшифрованный текст: vladislav
(.venv) dvvd@Debux:~/Documents/Crypto$
```

Рисунок 7 — Проверка корректности реализации шифрования и расшифрования шифра Хилла

```
(.venv) dvvd@Debux:~/Documents/Crypto$ /home/dvvd/Documents/Crypto/.venv/bin/python /home/dvvd/Documents/Crypto/HillCipher/HillCipher.py
Пропишите '1', если по умолчанию следует использовать латинский строчный алфавит.
Введите алфавит: 1
Введите текст: vladislav
Введите длину блока разбиения текста: 3
Введите '1' для обычного шифра Хилла, '2' для рекуррентного: 2
Введите матрицу первого ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 1 2 3
Строка 2: 0 1 4
Строка 3: 5 6 0
Введите матрицу второго ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 2 1 1
Строка 2: 1 1 0
Строка 3: 1 1 1
Введите '1' для шифрования, '2' для расшифрования: 1

Зашифрованный текст: rlpgldbcf
(.venv) dvvd@Debux:~/Documents/Crypto$ /home/dvvd/Documents/Crypto/.venv/bin/python /home/dvvd/Documents/Crypto/HillCipher/HillCipher.py
Пропишите '1', если по умолчанию следует использовать латинский строчный алфавит.
Введите алфавит: 1
Введите текст: rlpgldbcf
Введите длину блока разбиения текста: 3
Введите '1' для обычного шифра Хилла, '2' для рекуррентного: 2
Введите матрицу первого ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 1 2 3
Строка 2: 0 1 4
Строка 3: 5 6 0
Введите матрицу второго ключа построчно (числа через пробел, нужно ввести 3 строк(и)):
Строка 1: 2 1 1
Строка 2: 1 1 0
Строка 3: 1 1 1
Введите '1' для шифрования, '2' для расшифрования: 2

Расшифрованный текст: vladislav
```

Рисунок 8 — Проверка корректности реализации шифрования и расшифрования рекуррентного шифра Хилла

Результаты сошлись.

5 Примеры криптоанализа

В обоих примерах криптоанализа на основе открытого текста используется строчный латинский алфавит. *Шифр Хилла*: Шифруем слово «mathematicstralalelotralala». Используем ключ

$$K = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} \tag{15}$$

Результатом шифрования является «ryizahkzkrqoyshibosmjyshwlo». Пусть нам известно, что «mathemati» соответствует «ryizahkzk». Для того, чтобы декодировать текст, я реализовал метод вычисления обратной матрицы по модулю 26.

```python
import numpy as np

alphabet = "abcdefghijklmnopqrstuvwxyz"
char_to_num = {char: idx for idx, char in enumerate(alphabet)}
num_to_char = {idx: char for idx, char in enumerate(alphabet)}
```

[19] ✓ 0.0s Python

```python
def mod_inverse_3x3(matrix, modulus=26):
    """Находит обратную матрицу 3x3 по модулю."""
    M = np.array(matrix, dtype=int)

    det = int(np.round(np.linalg.det(M))) % modulus

    det_inv = None
    for i in range(modulus):
        if (det * i) % modulus == 1:
            det_inv = i
            break

    if det_inv is None:
        raise ValueError(f"Детерминант {det} не имеет обратного элемента по модулю {modulus}.")


    adj = np.round(np.linalg.inv(M) * np.linalg.det(M)).astype(int)

    inv_matrix = (det_inv * adj) % modulus
    return inv_matrix
```

[20] ✓ 0.0s Python

Рисунок 9 — Реализация функции вычисления обратной матрицы по модулю 26 и подготовка данных

```python
p_text = "mathemati"
c_text = "ryizahkzk"

p_nums = [char_to_num[c] for c in p_text]
c_nums = [char_to_num[c] for c in c_text]

P = np.array([
    [p_nums[0], p_nums[3], p_nums[6]],
    [p_nums[1], p_nums[4], p_nums[7]],
    [p_nums[2], p_nums[5], p_nums[8]]
])

C = np.array([
    [c_nums[0], c_nums[3], c_nums[6]],
    [c_nums[1], c_nums[4], c_nums[7]],
    [c_nums[2], c_nums[5], c_nums[8]]
])

P_inv = mod_inverse_3x3(P)


K = np.dot(C, P_inv) % 26

print("Матрица ключа K:\n", K)
```

21] ✓ 0.0s Python

```
Матрица ключа K:
 [[1 2 3]
 [0 1 4]
 [5 6 0]]
```

Рисунок 10 — Подсчет ключа

Получив ключ, можно расшифровать весь текст.

Криптоанализ рекуррентного шифра Хилла. Ключ

$$K_1 = \begin{pmatrix} 1 & 2 \\ 3 & 5 \end{pmatrix}, K_2 = \begin{pmatrix} 4 & 7 \\ 3 & 1 \end{pmatrix} \tag{16}$$

Текст - «mathematicstralala» Для подбора ключей реализована следующая функция:

```python
alphabet = "abcdefghijklmnopqrstuvwxyz"


p_text = "mathematicst"
c_text = "mkvmsaitysvi"

P = [char_to_num[c] for c in p_text]
C = [char_to_num[c] for c in c_text]

P_blocks = [np.array(P[i:i+2]).reshape(-1, 1) for i in range(0, len(P), 2)]
C_blocks = [np.array(C[i:i+2]).reshape(-1, 1) for i in range(0, len(C), 2)]

possible_k1 = []
for a in [1, 14]:
    for c in [3, 16]:
        for b in range(26):
            for d in range(26):
                det = (a * d - b * c) % 26
                if math.gcd(det, 26) == 1:
                    possible_k1.append(np.array([[a, b], [c, d]]))

possible_k2 = []
for x in range(26):
    y = (x + 3) % 26
    for z in range(26):
        w = (z + 24) % 26
        det = (x * w - y * z) % 26
        if math.gcd(det, 26) == 1:
            possible_k2.append(np.array([[x, y], [z, w]]))

valid_keys = []
for k1 in possible_k1:
    for k2 in possible_k2:
        k3 = np.dot(k2, k1) % 26
        if np.array_equal(np.dot(k3, P_blocks[2]) % 26, C_blocks[2]):
            k4 = np.dot(k3, k2) % 26
            if np.array_equal(np.dot(k4, P_blocks[3]) % 26, C_blocks[3]):
                k5 = np.dot(k4, k3) % 26
                if np.array_equal(np.dot(k5, P_blocks[4]) % 26, C_blocks[4]):
                    k6 = np.dot(k5, k4) % 26
                    if np.array_equal(np.dot(k6, P_blocks[5]) % 26, C_blocks[5]):
                        valid_keys.append((k1, k2))

print(f"Найдено возможных пар ключей: {len(valid_keys)}\n")
for i, (k1, k2) in enumerate(valid_keys):
    print(f"--- Исходные ключи (Пара #{i+1}) ---")
    print(f"K1:\n{k1}\n\nK2:\n{k2}\n")
```

Рисунок 11 — Реализация функции для подбора ключей рекуррентного шифра Хилла

Первая пара ключей оказалось верной

```
Найдено возможных пар ключей: 2

--- Исходные ключи (Пара #1) ---
K1:
[[1 2]
 [3 5]]

K2:
[[4 7]
 [3 1]]

--- Исходные ключи (Пара #2) ---
K1:
[[14 15]
 [ 3  5]]

K2:
[[4 7]
 [3 1]]
```

Рисунок 12 — Проверка ключа

6 Выводы о проделанной работе

Шифр Хилла является примером применения методов линейной алгебры в криптографии. Основным преимуществом данного шифра является его устойчивость к частотному анализу, поскольку один и тот же символ может быть зашифрован по разному. Его рекуррентная версия обеспечивает еще более высокий уровень безопасности, генерируя уникальный ключ для каждого блока открытого текста, тем самым усложняя криптоанализ. Однако, оба шифра уязвимы к атаке на основе известных пар открытого текста и соответствующего ему шифротекста, что позволяет злоумышленнику восстановить ключ и расшифровать сообщения, зная лишь n^2 пар символов, соответствующих открытому тексту. Рекуррентный шифр Хилла усложняет подсчет исходных ключей, однако не является абсолютно устойчивым к этому метод криптоанализа.

7 Список использованных источников

1. CUCCCUMBA. Обзор шифра Хилла [Электронный ресурс]. 2023. URL: https://habr.com/ru/articles/710890/ (дата обращения: 10.03.2026).